Day 4: Introduction to Kubernetes & GKE

---

1. Kubernetes Architecture

Definition

Kubernetes is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications.

Core Components

1. Pod

- Smallest deployable unit in Kubernetes
- Contains one or more containers

Example:

- A Pod running a Node.js application

---

2. Deployment

- Manages Pods and ensures desired state
- Handles scaling and updates

Example:

- Maintain 3 replicas of an application

---

3. Service

- Exposes Pods to internal or external traffic
- Provides stable networking

Types:

- ClusterIP (internal)
- NodePort (external access via node)
- LoadBalancer (cloud-based external access)

Architecture Overview

- Control Plane: API Server, Scheduler, Controller Manager
- Worker Nodes: Run Pods

Scenario

Application downtime due to server failure

Problem:

- Application hosted on single server

Solution:

- Deploy using Kubernetes Deployment

Result:

- Auto-healing and high availability

Case Study

Streaming Platform

- Uses Kubernetes for scaling services

Result:

- Automatically scales during peak traffic
- Ensures zero downtime

---

2. Introduction to GKE (Google Kubernetes Engine)

Definition

GKE is a managed Kubernetes service that simplifies cluster creation, management, and scaling.

GKE Modes

1. Standard Mode

- Full control over nodes
- Manual configuration

2. Autopilot Mode

- Fully managed nodes
- Pay per Pod
- Automatic scaling and security

Comparison

Feature	Standard	Autopilot
Node Management	User-managed	Fully managed
Flexibility	High	Limited
Cost Model	VM-based	Pod-based
Maintenance	Manual	Automatic

Scenario

Startup without DevOps team

Problem:

- No expertise in managing nodes

Solution:

- Use Autopilot mode

Result:

- Reduced operational overhead

Case Study

E-commerce Startup

- Uses GKE Autopilot

Result:

- Focus on development instead of infrastructure
- Reduced operational cost

---

3. Resource Requests, Limits, and Health Probes

Resource Requests and Limits

Definition

- Request: Minimum CPU/memory guaranteed
- Limit: Maximum CPU/memory allowed

Example

```
resources:
  requests:
    memory: "128Mi"
    cpu: "250m"
  limits:
    memory: "256Mi"
    cpu: "500m"
```

Scenario

Application consuming too much memory

Problem:

- One container affects others

Solution:

- Set resource limits

Result:

- Stable cluster performance

Health Probes

Types

1. Liveness Probe

- Checks if app is alive
- Restarts container if failed

2. Readiness Probe

- Checks if app is ready to serve traffic

Example

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Scenario

Application crashes silently

Solution:

- Use liveness probe

Result:

- Automatic restart

Case Study

Banking Application

- Uses readiness probes

Result:

- Prevents traffic to unhealthy Pods

---

4. Namespaces, ConfigMaps, and Secrets

Namespace

Definition

Logical separation of resources within a cluster

Example:

- dev
- test
- prod

Scenario

Multiple teams using same cluster

Solution:

- Use namespaces

Result:

- Isolation and better management

ConfigMap

Definition

Stores non-sensitive configuration data

Example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  ENV: production
```

Secrets

Definition

Stores sensitive data (passwords, API keys)

Example:

```
apiVersion: v1
kind: Secret
```

```
metadata:
  name: db-secret
type: Opaque
data:
  password: cGFzc3dvcmQ=
```

Scenario

Database credentials exposed in code

Solution:

- Use Secrets

Result:

- Secure application configuration

Case Study

Enterprise Application

- Uses ConfigMaps for configs
- Uses Secrets for credentials

Result:

- Secure and flexible deployments

---

5. Hands-On: Deploy Container to GKE with YAML

Step 1: Create Cluster

```
gcloud container clusters create my-cluster --zone asia-south1-a
```

Step 2: Get Credentials

```
gcloud container clusters get-credentials my-cluster --zone asia-south1-a
```

Step 3: Deployment YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: myapp
        image: <image-url>
        ports:
        - containerPort: 8080
```

Step 4: Service YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: LoadBalancer
  selector:
    app: myapp
```

```
  ports:
  - port: 80
    targetPort: 8080
```

Step 5: Apply YAML

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Step 6: Verify

```
kubectl get pods
kubectl get services
```

End-to-End Case Study (Day 4 Summary)

Scenario: Scalable Web Application Deployment

Problem:

- Application downtime
- No scalability
- Hard to manage infrastructure

Solution

1. Deploy using Kubernetes Deployment
2. Expose using Service
3. Use GKE for managed infrastructure
4. Configure resources and probes
5. Use namespaces and secrets

Result

- High availability
- Auto-scaling capability

- Secure configuration
- Efficient resource utilization